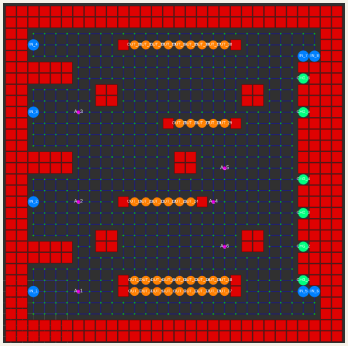
Scalable Multi-Robot Coordination

Integrating LCBA Task Allocation with Decentralized Path Planning



Shreyas Raorane

MSE Robotics · University of Pennsylvania · May 2026

Thesis Committee

Dr. Linh Thi Xuan Phan
Dr. Rahul Mangharam
Dr. M. Ani Hsieh

Presentation Outline

01

Motivation

Warehouse coordination breaks at scale

02

Background & Problem

CBBA, GCBBA, and where they fail

03

LCBA Algorithm

Three key relaxations; convergence guarantees

04

System Architecture

Modular pipeline; path planners

05

Experimental Setup

432+ runs, 6 environments, dual evaluation modes

06

Results & Conclusions

Batch · Steady-state · Efficiency · Path planners

Modern Warehouses: A Coordination Challenge

The Scale Problem

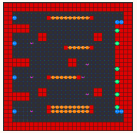
- **Amazon/Ocado**: 1,000s of robots on one floor. New tasks arrive every few seconds.
- **Coordination** requires solving:
  - *Assignment*: Who picks which item?
  - *Pathing*: Collision-free trajectories.
  - *Timing*: When to revise decisions?

The Real-World Constraint:
Range-limited radios + metal shelving ⇒ **intermittently disconnected** networks. Existing algorithms silently fail here.

Evaluation Environments

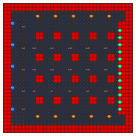
Maps spanning from small-fleet precision to high-density coordination.

Warehouse Small



30×30 · 6 agents

Kiva



36×36 · 20 agents

Multi-Robot Task Allocation: Two Paradigms

Centralized

- All task/agent state gathered at a single planner.
- **SGA** assigns highest-marginal-gain (agent, task) pairs greedily; ≥50% optimal in poly time.
- **Weaknesses:** single point of failure; requires reliable broadcast; scales poorly.

Decentralized (Auction-Based)

- Agents bid using local info; conflicts resolved via message-passing consensus.
- **CBBA** (Choi 2009): pairwise resolution; converges to 50%-optimal on *connected* graphs.
- **GCBBA** (Kha 2025): $2D$ consensus rounds; converges to SGA on connected graphs.

Shared assumption: Both CBBA and GCBBA require the communication graph to be CONNECTED. When disconnected, both produce conflicting assignments and leave agents idle.

Where CBBA and GCBBA Break

GCBBA Disconnection Failures

- Independent consensus $\Rightarrow$ conflicting claims.
- $D = \infty$ in disconnected graphs $\Rightarrow$ never terminates.
- No reconnection protocol or online arrivals.

Performance Cliff (Experimental)

CBBA: Fails/times out on 64–96 tasks ($r_{\text{comm}} = 4, 25$).

DMCHBA (SOTA): Makespan **nearly 5× slower** when disconnected (2725 vs 515 ts).

GCBBA Algorithm Structure

1

Bundle Construction
Greedily insert tasks by highest utility.

2

Consensus
Broadcast bids; $2D$ rounds for convergence.

3

Convergence
Matches SGA solution on connected graphs.

The status quo. GCBBA solves task allocation on fully connected graphs with guaranteed optimality. **But all three phases break when the communication graph fragments.** This thesis extends these phases to handle disconnection gracefully.

LCBA: Three Key Relaxations over GCBBA

01 Component-Local Consensus

Bidding and $2d_k$ rounds operate within each connected component k (diameter d_k). No global connectivity required; always terminates.

02 Timestamped Conflict Resolution

On component merge, agents exchange timestamped bid tables. Stale claims overwritten; displaced agents re-enter bundle building.

03 Event-Driven Replanning

Tasks arrive continuously. Five event triggers fire LCBA without waiting for a fixed schedule.

Theoretical guarantee. On connected components, LCBA inherits GCBBA's convergence and greedy-style **allocation-utility** guarantee (worst-case $\leq 2\times$ from optimal under standard assumptions).

Research Questions

- **RQ1** Can LCBA maintain task throughput under communication conditions where static allocation fails?
- **RQ2** How close does LCBA stay to the SOTA reference (DMCHBA) as communication connectivity varies?
- **RQ3** Does LCBA's robustness generalize across diverse warehouse environments and scales?

LCBA Algorithm: Bundle Construction

Phase 1: Bundle Construction

- Each agent i maintains a **bundle** b_i (ordered task list) and **path** p_i.
- Greedy insertion: scan all unassigned tasks; insert the task j^* that maximizes marginal **RPT utility**

$$\Delta c_{ij} = c_i(b_i \cup \{j\}) - c_i(b_i).$$

- **Energy-budget filter:** skip task j if the agent cannot reach j and then a charger before battery depletes.
- Repeat until no beneficial insertion or $|b_i| = L_{\max}$.

Practical note

Bundle construction is lightweight and bounded by the bundle length $L_{\max}$.

Tip. Early termination in insertion reduces compute by avoiding full-scan marginal gains when no beneficial insertions remain.

LCBA Algorithm: Local Consensus

Phase 2: Local Consensus

- Agents share winning bid tables with neighbors *within their connected component*.
- Apply update / reset / leave rules; run $2d_k$ rounds (d_k = component diameter).
- Conflicts resolved by **highest bid**; ties broken by agent ID.

Efficiency gain. At 18 agents, minimum r_{comm}: CBBA runs 31.6 rounds (47 ms); LCBA runs 15.8 rounds (15 ms) — a **3× speedup** per allocation call.

Why component-local consensus is sufficient

- Agents in different components cannot coordinate anyway.
- Running $2d_k$ rounds (not $2D$ over the full graph) guarantees convergence *within* each component.
- At full connectivity ($d_k = D$), LCBA is identical to GCBBA — no overhead.

LCBA Algorithm: Reconnection & Event-Driven Replanning

Phase 3: Reconnection Handling

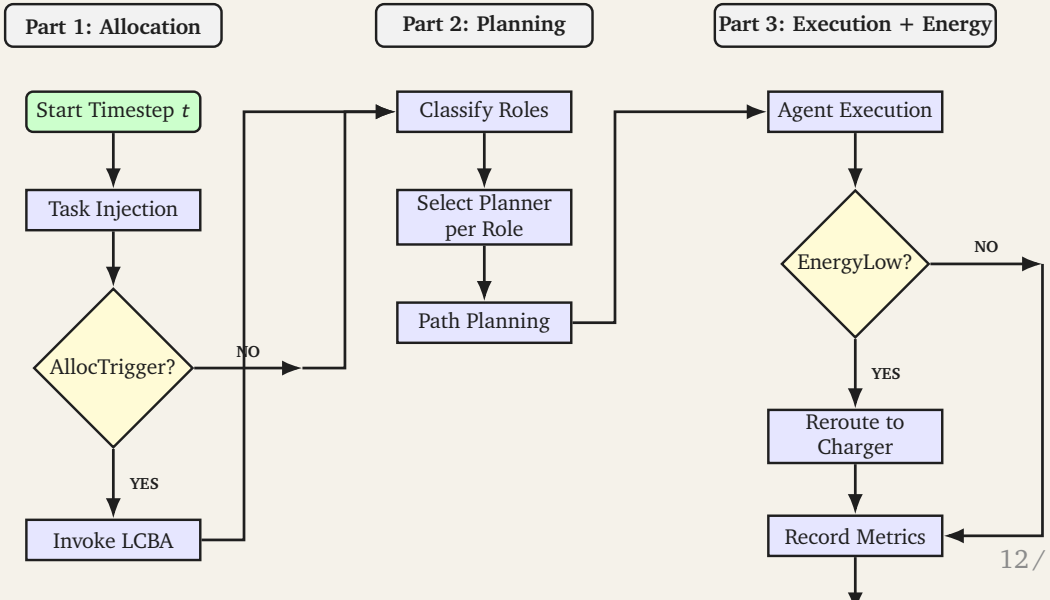
- The communication graph is reconstructed every timestep as agents move.
- When two components k and k' merge, agents exchange full bid tables.
- Conflict resolution: newer timestamp wins; same timestamp $\Rightarrow$ higher bid wins.
- Agents whose tasks are displaced by the merge re-enter bundle construction *for those tasks only*.
- **Result:** no task is permanently lost during topology changes.

LCBA Event-Driven Replanning Triggers:

1. $t = 0$ or restart.
2. Charging state changes.
3. New tasks arrive AND idle agents exist.
4. $\geq \lceil N_a/3 \rceil$ completions since last run AND cooldown done.
5. Pending tasks remain AND idle agents exist AND cooldown done.

Why event-driven? Fixed-interval replanning either wastes compute at low load or misses surges at high load. Triggers 3–5 adapt naturally to task rate.

System Architecture: Full Coordination Pipeline



Operational Safeguards

Proactive Charging

Agents navigate to chargers before battery depletion.

Energy-Budget Filter

LCBA skips bids on tasks an agent cannot complete before needing to charge.

Wait Stations

Idle agents park off main aisles to keep active picking corridors clear.

Experimental Setup

Parameter Sweep

- **432+ runs** across all configurations.
- **4 algorithms:** LCBA, CBBA, SGA, DMCHBA.
- **6 communication ranges:** fully isolated → fully connected (diagonal fractions $\{0.1, 0.2, 0.35, 0.6, 1.2, \infty\}$).
- **3 path planners:** CA*, PBS, RHCR.
- **12 arrival rates** (steady-state): 0.25× – 1.4× estimated capacity.
- **2 random seeds:** 42 and 123.

Evaluation Modes

Batch

Fixed task set; primary metric = completion rate; secondary = completed-only makespan.

Steady-State

Continuous arrivals; throughput over 1500 ts; Metrics: throughput, mean latency, 95%-latency, completion rate, agent utilization.

Baselines

CBBA

Decentralized auctions with pairwise consensus. Local decisions; robust to partial connectivity but may fragment into components.

SGA

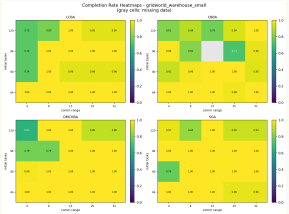
Centralized greedy reference. High performance when fully connected; degrades to per-component greedy behavior under disconnection.

DMCHBA

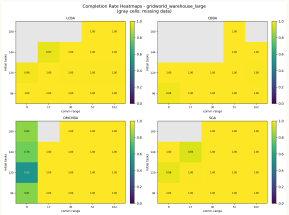
State-of-the-art decentralized matching; uses local Hungarian matching per component for near-optimal assignments locally.

Batch Results: Completion Rate Heatmaps

Warehouse Small · 6 agents



Warehouse Large · 18 agents



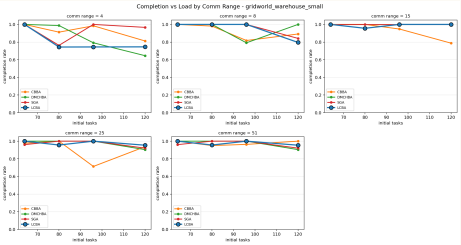
LCBA. (top-left) 100% completion at *all* comm ranges at light load; reliable from $r_{\text{comm}} = 8$ upward at moderate load.

CBBA. (top-right) Fails on both seeds at $r_{\text{comm}} = 4$ and $r_{\text{comm}} = 25$ at 96 tasks; timeouts grow sharply with load.

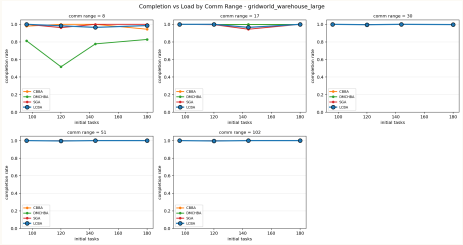
DMCHBA. (bottom-left) ≈2725 ts at $r_{\text{comm}} = 4$ vs. 515 ts connected.

Batch Results: Completion Degradation by Comm Range

Warehouse Small



Warehouse Large



Each line is a communication range. Flat lines indicate graceful handling of load; steep drops indicate a performance cliff. **LCBA lines are consistently flat; CBBA and SGA show steep cliffs at low r_{comm}.**

Batch Results: Makespan and Sub-Optimality (RQ2)

Completed-only makespan at light load (64 tasks, Warehouse Small)

Method	$r_{\text{comm}} = 4$ *(isolated)*	$r_{\text{comm}} \geq 25$ *(connected)*
LCBA	685 ts	640 ts
SGA	663 ts	519 ts
CBBA	860 ts	640 ts
DMCHBA	2725 ts	515 ts

RQ2 answer. On connected graphs, LCBA is within the same batch-time range as the SOTA reference (DMCHBA) on the connected case, while degrading much more gracefully as connectivity drops.

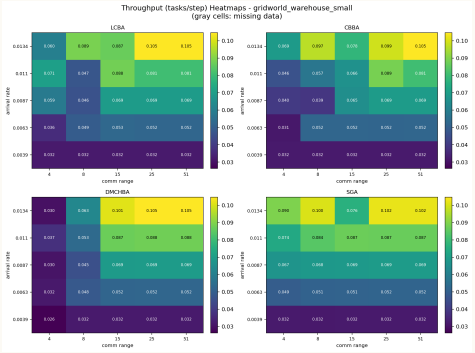
Key observations

- LCBA degrades from 640 to 685 ts as r_{comm} drops: **only** 7%.
- CBBA degrades from 640 to 860 ts: **34%**.
- DMCHBA suffers most: **430%** at fully disconnected because matching quality degrades as the graph fragments.
- SGA timed out on seed 42; per-component SGA is an imperfect reference when disconnected.

Takeaway. LCBA achieves near SOTA quality on connected graphs and degrades much more gracefully than baselines as connectivity drops.

Steady-State: Throughput Heatmap (Warehouse Small)

Throughput (tasks/ts) vs. arrival rate & comm range



Warehouse Small (30×30, 6 agents)

Heatmap

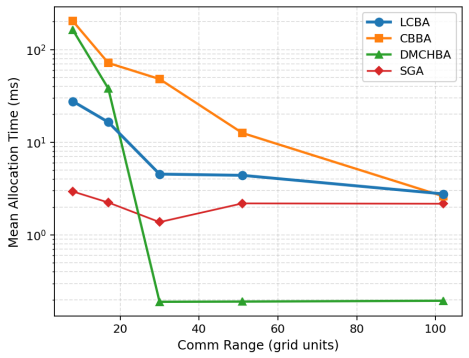
- Rows = arrival rate (λ); cols = comm range (r_{comm}).
- Each method is a separate panel.

Key findings

- **LCBA:** near-uniform brightness across all comm ranges at each arrival rate.
- **DMCHBA:** drops sharply to 0.017 tasks/ts at $r_{\text{comm}} = 4$ (vs. 0.067 connected)
- **CBBA:** moderate drop at $r_{\text{comm}} = 8$ (0.039 vs. 0.046 tasks/ts).
- At full connectivity, all methods reach the same throughput ceiling — **LCBA adds no overhead**.

Allocation Efficiency: Speed Across the Spectrum

Mean allocation time vs. comm range



Warehouse Large (60×60, 18 agents)

Why LCBA is faster at low r_{comm}

- CBBA runs $N_{\min} \times D$ rounds where D is the *full graph diameter* — independent of fragmentation.
- LCBA runs $2d_k$ rounds where d_k is the *component diameter* — smaller when the graph is broken.
- At 18 agents: CBBA = 31.6 rounds (47 ms) vs. LCBA = 15.8 rounds (15 ms) $\Rightarrow$ **3× speedup**.

LCBA vs. DMCHBA

- DMCHBA clones the full task list and runs Hungarian matching independently per component.
- Overhead grows multiplicatively with component count; LCBA avoids this.

Conclusions: Contributions

Five Contributions

01 LCBA Algorithm
Component-local consensus + timestamped reconnection + event-driven replanning; inherits the $\leq 2\times$ allocation-utility worst-case guarantee.

02 Hybrid Pipeline
Modular allocation + 3 interchangeable path planners; layers independently configurable.

03 Dual-Mode Simulation
Batch (completion, makespan) and steady-state (throughput, wait, queue depth) in one environment.

05 Operational Realism
Proactive charging, energy-budget bid filter

RQ1. 100% completion at all comm ranges at light load; maintains throughput where baselines fail.

RQ2. On the connected case, LCBA is close to the SOTA baseline (DMCHBA), and it is much more stable under disconnection.

RQ3. Results generalize across 6 diverse environments (6–200 agents, 30×30 to 161×63); consistent robustness at all scales.

Broader applicability. Any domain where communication reliability cannot be guaranteed.

Limitations & Future Work

Acknowledged Limitations

- **Centralized path planning:** shared space-time reservation table is a single point of failure.
- **Simulation only:** discrete gridworld does not capture sensor noise, actuation uncertainty, curved aisles, or human pedestrians.
- **Per-component SGA reference** is conservative on disconnected graphs — true global optimal is higher.
- **Timestep ceiling:** worst-case data at extreme load + min r_{comm} reflects timeouts, not converged failure.

Future Work

- **Fully decentralized collision avoidance:** per-component CBS, distributed RHCR, or PIBT to remove the central reservation table.
- **Heterogeneous fleets:** mixed speeds, payloads, battery capacities.
- **Scaling beyond 200 agents:** real warehouse floor plans from operational data.
- **Hardware validation:** basic deployment on some hardware.

Thank You

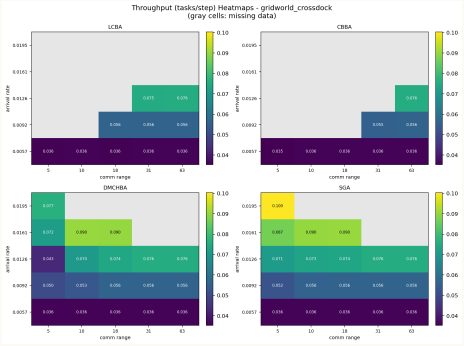
Questions?

Scalable Multi-Robot Coordination:
Integrating LCBA Task Allocation with Decentralized Path Planning

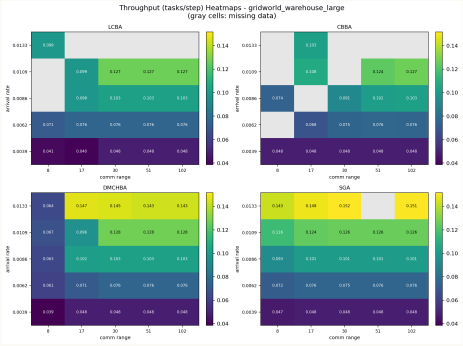
Shreyas Raorane · MSE Robotics, University of Pennsylvania
Dr. Linh Thi Xuan Phan · Dr. Rahul Mangharam · Dr. M. Ani Hsieh

Backup: Steady-State Throughput — Crossdock & Warehous
Large

Crossdock · 12 agents

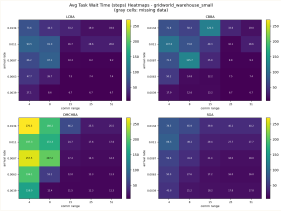


Warehouse Large · 18 agents

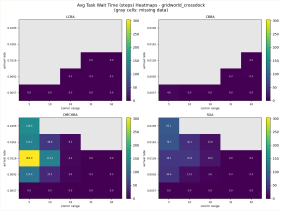


Backup: Steady-State Wait-Time Heatmaps

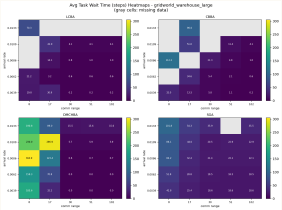
Warehouse Small



Crossdock



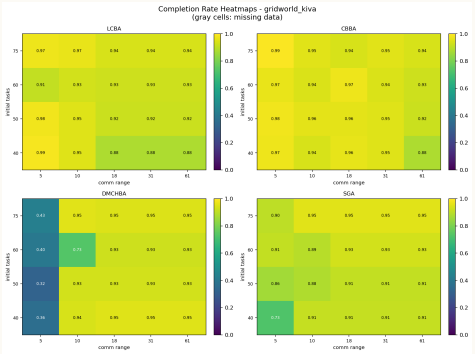
Warehouse Large



Wait time = timesteps from task injection to first agent claim. Lower is better. LCBA maintains low wait time across the full comm spectrum; baselines incur high wait time under degraded connectivity due to delayed or failed re-assignment.

Backup: Kiva Batch Completion Heatmap (Secondary Map)

Kiva · 36×36 · 20 agents



Secondary / partial-completion map

Context

- Kiva is classified as *secondary* because narrow aisles cause timeout-dominated behavior at moderate- to-high loads.
- Kiva results illustrate robustness at the boundary of reliable operation.
- LCBA maintains non-zero completion across all ranges even in this stress-adjacent regime.

Backup: Path Planner Comparison

All three planners maintain zero vertex conflicts across all tested runs on Warehouse Small (verified by post-hoc safety audit).

Performance Characteristics

- **CA*** (Cooperative A*) — plans all agents jointly each call. Best path quality; highest compute. Comparable to PBS at small scale ($N_a \leq 12$).
- **PBS** (Priority-Based Search) — prioritized sequential replanning. Similar quality and cost to CA* at small-to-medium scale.
- **RHCR** (Rolling Horizon Collision Resolution) — fixed-horizon window; most bounded per-call

Allocation timing (Warehouse Small, light load, connected)

Method	Avg. alloc time (ms)
LCBA	1.0–5.5
CBBA	1.2–8.4
SGA	0.3–0.6
DMCHBA	0.04–0.05

No single planner dominates. CA* is best for small fleets;

Backup: Steady-State Metric Definitions

Metric Definitions

- **Throughput** = steady-state tasks completed ÷ (1500 − 300) timesteps.
- **Warmup**: first 300 timesteps excluded (system settling).
- **Wait time**: timesteps from task injection to first agent claim.
- **Queue depth**: per-station pending task count; capped at 5; excess arrivals counted as dropped.
- **Capacity frontier**: highest λ sustaining ≥95% of peak throughput.

Task Arrival Model

- Deterministic: one task per induct station

Arrival rate sweep (across modes)

Mode	Fractions of capacity
Full	$\{0.25, 0.4, 0.55, 0.7, 0.85\}$
Quick	$\{0.5, 0.8\}$
Stress	$\{0.6, 0.8, 1.0, 1.2, 1.4\}$

Batch Parameters

- Batch tasks per induct (full): $\{8, 10, 12, 15\}$.
- Batch tasks per induct (stress):